

# Desafío Técnico – Desarrollador Full-Stack (Mid / Senior)

**Tiempo sugerido:** 3 a 5 días naturales (dedicación parcial) **Idioma de entrega:** Español

**Entrega:** URL de repositorio público + instrucciones para `docker compose up --build`

---

## Contexto

Una startup ficticia llamada **Deskly** necesita un prototipo funcional de **sistema de tickets de soporte**. El producto debe permitir:

1. Registrar y gestionar tickets con un ciclo de vida controlado (máquina de estados).
2. Recibir tickets desde sistemas externos vía **webhook firmado**.
3. Notificar cambios en **tiempo real** a los agentes conectados vía **WebSocket**.
4. Exponer una vista pública con **SSR** para consulta de un ticket por su identificador.

El equipo usa Docker, FastAPI y Next.js. La base de datos puede ser **PostgreSQL** o **MongoDB** (tu elección — justifícala).

---

## Política de uso de LLMs (lee esto antes de empezar)

El uso de LLMs (Claude, ChatGPT, Copilot, Cursor, etc.) está **permitido sin límite de alcance**. No nos interesa saber si escribiste el código a mano.

**Lo que sí evaluamos es tu razonamiento, no el del modelo.** Por lo tanto:

- Todo uso relevante de LLM debe estar **fundamentado**: qué le pediste, por qué, qué te devolvió y **qué decidiste tú** sobre esa salida (aceptar, modificar, descartar).
- Documenta esto en un archivo `DECISIONES.md` (ver formato abajo). No es un anexo opcional: **pesa en la evaluación**.
- Si aceptaste una sugerencia sin entenderla, dilo. Preferimos honestidad a una justificación inventada.
- **En la entrevista de seguimiento te pediremos defender cualquier línea del repositorio**. No poder explicar tu propio código pesa más negativamente que una funcionalidad incompleta.

Regla práctica: si el LLM tomó una decisión de arquitectura por ti y no puedes explicar la alternativa que descartó, esa decisión aún no es tuya.

---

## Tu misión

**Backend (FastAPI)**

**Obligatorio**

- Modelo `Ticket` con al menos: `id`, `titulo`, `descripcion`, `prioridad`, `estado`, `asignado_a`, `creado_en`, `actualizado_en`.
- Endpoints REST:
  - `POST /api/tickets` → crear
  - `GET /api/tickets` → listar con **paginación y filtros** (estado, prioridad, asignado)
  - `GET /api/tickets/{id}` → detalle
  - `PATCH /api/tickets/{id}` → actualizar campos
  - `POST /api/tickets/{id}/transicion` → cambiar estado
  - `POST /api/tickets/{id}/comentarios` → añadir comentario
- **Máquina de estados** con transiciones válidas explícitas: `abierto → en_progreso → resuelto → cerrado`, con `reabierto` permitido desde `resuelto`. Una transición inválida debe devolver `409` con un cuerpo de error tipado, no un `500`.
- Validación con Pydantic v2. Esquemas de entrada y salida **separados**.
- **Migraciones versionadas** (Alembic si usas SQL; script de índices idempotente si usas Mongo).

## Webhook de ingesta

- `POST /api/webhooks/tickets` que crea tickets desde un sistema externo simulado.
- Verificación de firma **HMAC-SHA256** con secreto compartido, comparación en tiempo constante.
- Protección contra **replay** (ventana de tiempo sobre el timestamp del header).
- **Idempotencia**: el mismo `event_id` recibido dos veces produce un solo ticket.
- Firma inválida → `401`. Payload malformado → `422`. Evento duplicado → `200` sin efecto secundario.

## WebSocket

- `WS /ws/tickets` que emite eventos a los clientes conectados: `ticket.creado`, `ticket.actualizado`, `ticket.comentado`.
- Soporte de suscripción por ticket concreto o a todos.
- Gestión correcta del ciclo de vida: registro, desconexión limpia, sin fugas de conexiones muertas.
- Documenta en el README la **limitación con múltiples réplicas** y cómo lo resolverías en producción.

---

## Frontend (Next.js 14+, App Router)

### Obligatorio

- **Dashboard** (`/`): tabla de tickets con filtros y paginación, sincronizados con la URL (query params).
  - **Detalle** (`/tickets/[id]`) renderizado en **servidor (SSR)**, con hilo de comentarios y acciones de transición.
  - Conexión **WebSocket desde cliente** que actualice la UI en vivo sin recargar:
    - Hook propio (`useTicketStream`) o similar) con limpieza en (`useEffect`).
    - **Reconexión con backoff exponencial** e indicador visible de estado de conexión.
  - **Optimistic UI** en al menos una acción (transición de estado o comentario), con rollback en caso de error.
  - Renderizado condicional: estados de carga, vacío y error diferenciados. Nada de spinners genéricos para todo.
  - Tipado end-to-end: tipos compartidos o generados desde el esquema OpenAPI del backend.
- 

## Base de datos

- PostgreSQL o MongoDB, a tu elección.
  - **Índices justificados** para las consultas de listado y filtrado.
  - Justifica la elección en el README: por qué esa y no la otra, para este caso concreto.
- 

## DevOps

- `Dockerfile` **multi-stage** para backend y frontend. Imágenes razonablemente ligeras.
  - `docker-compose.yml` que levante backend, frontend y base de datos, con healthchecks.
  - Variables de entorno vía `.env.example`. **Ningún secreto en el repositorio.**
  - **GitHub Actions** con al menos:
    - Lint + type check (backend y frontend)
    - Tests unitarios
    - Al menos **un test de integración** contra base de datos efímera (service container)
    - Build de las imágenes Docker
    - El pipeline debe estar **en verde** en el repo entregado.
- 

## Testing

- Tests unitarios de la máquina de estados (transiciones válidas e inválidas).
- Test de verificación de firma del webhook (válida, inválida, replay, duplicado).
- Al menos un test de integración de un endpoint contra DB real.

- Frontend: al menos un test del hook de WebSocket o de un componente con estado.
- 

## Bonus (no obligatorio)

- Autenticación básica (JWT) con roles `agente` / `admin`.
  - Rate limiting en el endpoint de webhook.
  - Redis como pub/sub para que el WebSocket funcione con múltiples réplicas.
  - Búsqueda full-text nativa (`tsvector`) o índice de texto de Mongo).
  - Observabilidad: logs estructurados con `correlation_id` propagado entre webhook → DB → evento WS.
- 

## Reglas y restricciones

1. El arranque completo debe ser:

```
bash

git clone <tu-repo>
cd <repo>
cp .env.example .env
docker compose up --build
```

2. Sin dependencias de servicios externos de pago en tiempo de ejecución.
  3. Sin secretos versionados.
  4. Incluye un script de **seed** con datos de ejemplo.
  5. Commits atómicos con mensajes descriptivos. El historial forma parte de la evaluación.
- 

## Entrega

Repositorio público en GitHub con:

`README.md`

- Pasos de arranque y verificación
- Tiempo real invertido
- Elección de base de datos y por qué
- Diseño de la máquina de estados
- Estrategia de idempotencia y firma del webhook
- Limitación conocida del WebSocket ante múltiples réplicas
- Qué dejaste fuera y por qué

`DECISIONES.md` — mínimo 5 entradas, formato:

markdown

### [Decisión] Título breve

**\*\*Contexto:\*\*** qué problema estaba resolviendo.

**\*\*Uso de LLM:\*\*** qué le pedí y por qué (o "sin LLM").

**\*\*Salida del modelo:\*\*** resumen de lo que propuso.

**\*\*Mi decisión:\*\*** qué acepté, modifiqué o descarté, y con qué criterio.

**\*\*Alternativa descartada:\*\*** cuál era y por qué no.

Al menos **2 de las 5 entradas** deben describir un caso en el que **corregiste, rechazaste o rediseñaste** la propuesta del LLM.

## Criterios de evaluación

| Peso | Criterio |
|------|----------|
|------|----------|

|      |                                                                                       |
|------|---------------------------------------------------------------------------------------|
| 25 % | <b>Correctitud funcional</b> — endpoints, máquina de estados, webhook, WebSocket, SSR |
|------|---------------------------------------------------------------------------------------|

|      |                                                                                                       |
|------|-------------------------------------------------------------------------------------------------------|
| 20 % | <b>Calidad de código y arquitectura</b> — separación de capas, tipado, manejo de errores, legibilidad |
|------|-------------------------------------------------------------------------------------------------------|

|      |                                                                                           |
|------|-------------------------------------------------------------------------------------------|
| 15 % | <b>Razonamiento documentado</b> — <code>DECISIONES.md</code> y justificaciones del README |
|------|-------------------------------------------------------------------------------------------|

|      |                                                                                             |
|------|---------------------------------------------------------------------------------------------|
| 15 % | <b>Frontend</b> — hooks, estado, optimistic UI, renderizado condicional, resiliencia del WS |
|------|---------------------------------------------------------------------------------------------|

|      |                                                                                            |
|------|--------------------------------------------------------------------------------------------|
| 15 % | <b>Testing y CI/CD</b> — cobertura significativa, pipeline verde, test de integración real |
|------|--------------------------------------------------------------------------------------------|

|      |                                                                                               |
|------|-----------------------------------------------------------------------------------------------|
| 10 % | <b>Docker y modelado de datos</b> — compose funcional, imágenes limpias, índices justificados |
|------|-----------------------------------------------------------------------------------------------|

## Nota final

No buscamos que completes el 100 % del alcance. Buscamos ver **cómo priorizas, qué decides y por qué**. Un entregable parcial con criterio sólido y bien documentado vale más que uno completo que no puedas defender.

¡Éxito, y que el código —y tu razonamiento— hablen!